2. Difference b/w DFT engg and fn verification engg? Why do we need both DFT and fn ver engg?

Ans :- ① DFT enggs do not check the functionality of the chip. We rather check the quality of the chip.

DFT enggs insert test structures into the design during the pre-silicon stage (before man) to facilitate the detection of manufacturing defects in the chip.

Eg:- Half adder :-

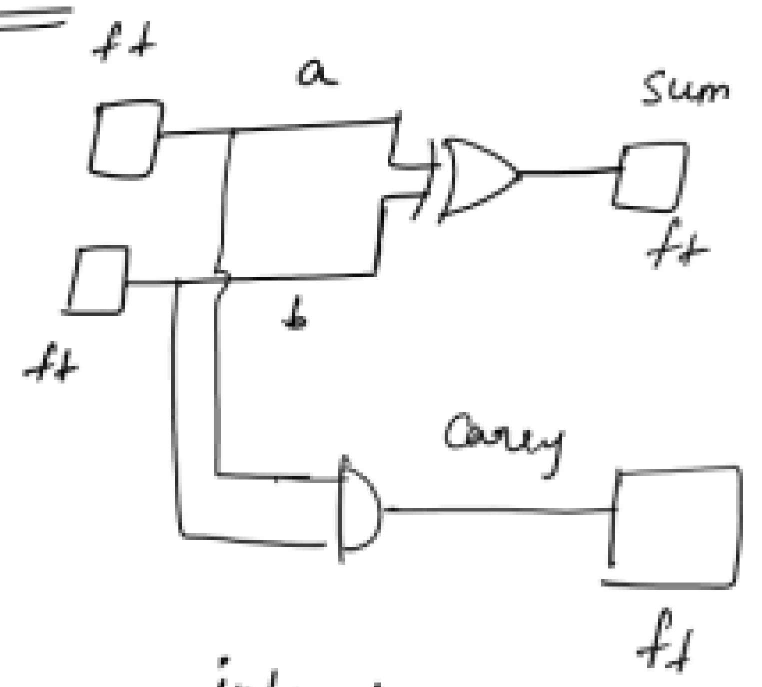
sum = a xor b
carry = ab } whether this is correct or not.

by a mistake, if someone has written

Sum = a + b
Carry = ab } this will not fn like a half adder.

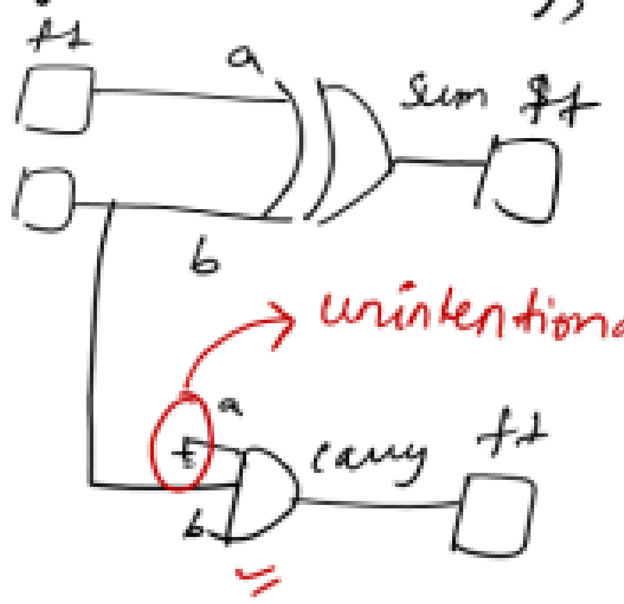
} checked by the fn verification
↳ checking the functionality.

DFT:- do not check the fn. Check whether each & every net of the ½ adder is free of defects or not.



intended design.

During manufacturing,



unintentionally during man, A net of this AND gate has got connected to gnd.

② DFT enggs require less test patterns to test a chip.

↳ optimised set of patterns is generated during ATPG.

(3) DFT enggs have more controllability and observability on the design. We take less time to test the chip.

Example :-

Design → 10 bit counter (0 to 1023)
counter init
counter obs

If suppose counter is failing (after 999, counter counts 999 instead of 1000)

Functional verification:-

- check the functionality.

In this case, the counter is failing at the 1000^{th} pulse.
So 1000 pulses are required.

- Some clock pulses are required for counter init and counter observation.

∴ Fn verification engg requires more than 1000 clk pulses to detect that the counter is failing

DFT engg :-

Require approximately in the range of 100 clk pulses to tell that the counter is failing.

DFT engg :- Do not check the fn.

We check whether each and every net is having any manufacturing defect or not.

↳ we don't write patterns to check the fn. We write patterns to check whether there are any manufacturing defects or not.

↳ For that we try to toggle (control) each and every net of the design to an appropriate value and observe the value.

∴ We will assume that we approximately require 6 to 7 patterns.

$\Rightarrow$ Suppose if we have given max length of scanchain = 10 → no. of ffs / scanchain

$P_1L = 10$ cycles
$P_1C = 1$ cycle
$P_1U, \underline{P_2L} = 10$ cycles
$\overline{21}$

$P_2C = 1$ cycle
$P_2U, P_3L = 10$ cycles
$\overline{11}$

$P_3C = 1$ cycle
$P_3U, \underline{P_4L} = 10$ cycles
$\overline{11}$

$P_4C = 1$ cycle
$P_4U, P_5L = 10$ cycles
$\overline{11}$

$P_5C = 1$ cycle
$P_5U, P_6L = 10$ cycles
$\overline{11}$

$P_6C = 1$ cycle
$P_6U, P_7L = 10$ cycles
$\overline{11}$

$P_7C = 1$ cycle
$P_7U = 10$ cycles
$\overline{11}$

Total $\Rightarrow$ 87 cycles → this can slightly vary based on the no. of patterns & the scan configuration.

$\therefore$ DFT engg will be able to tell that the counter is failing approx in the range of 100 clk pulses.

(4) Functional verification engg will be able to tell whether the chip is passing or failing.

DFT engg can additionally tell what manufact- -uring defect has caused the chip to fail.

Counter eg additional explanation :-

Suppose,

only functional verification team is there.

Counter ⇒ manufactured
⇓
Counter is having some defect
⇓
Counter is tested with fn patterns only
⇓
in that eg, it will take more than 1000 pulses to tell that the counter is failing and

fn verification team can only tell that the counter is failing.

↳ Reason :- They check the functionality. i.e; check whether the counter properly counts from 0 to 1023. After 999, it is again counting 999. But expected is, it should count 1000

∴ Failing.

Now consider we are having DFT team,

Counter ⇒ manufactured

⇓

counter is having some defect

⇓

DFT patterns are used for testing

⇓

we can tell that the counter is failing approximately in the range of 100 clk cycles itself

\+ we can tell what manufacturing defect has caused the counter to fail.

↳ How?

Go for scaninsertion : shift ⇒ we shift in i/pseq

capture ⇒ response of combo will be captured into the flop.

shift out ⇒ the captured value will be shifted out & we compare it with expected value.

If suppose there was any defect in the combo, then wrong value will be captured.

i/p seq & expected o/p

↳ Generated by ATPG tool

↳ Using Fault simulation

⇓

It will mimic all possible failure situations that might occur after manufacturing. It will try to find a pattern that will be able to detect the particular failure situation if suppose it occurs after manufacturing. And those patterns will be written into the final pattern list.

As DFT team generates patterns using fault simulation, if a particular pattern fails, we will be able to tell the possible manufacturing defects that would have occurred.

Summary :-

In this eg,

- if functional patterns are used to test the chip after man: more clockcycles are required to test the chip ⇒ Test time will be more

  and they can only tell if the chip has passed or failed.

- if DFT patterns are used to test the chip after man: less clock cycles are required to test the chip ⇒ Test time will be less

  and we can tell if the chip has passed or failed and we can also tell what manufacturing defect has caused the chip to fail.

As DFT patterns has these advantages over functional patterns, DFT patterns will be used to test the chip after manufacturing. Some functional patterns will also be used to test the chip after manufacturing.

Q:- If DFT patterns are used to test the chip after manufacturing, why do we need functional verification team?

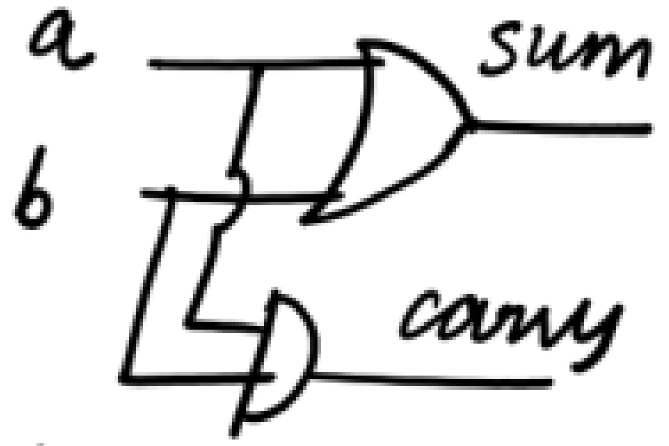
Ans:- Consider the below scenario:

half adder:-

$$\left.\begin{aligned} sum &= a + b; \\ carry &= ab \end{aligned}\right\}$$ functional bug

↓

synthesized



wrong design

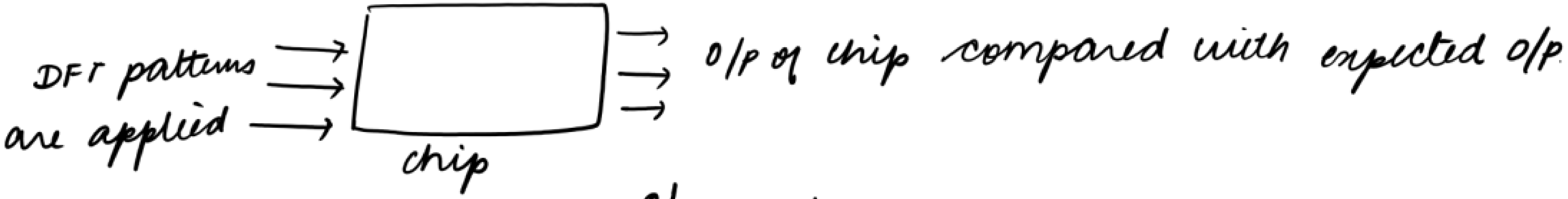
↓

DFT :- DFT will generate patterns just to check whether there are any manufacturing defects in each and every net of this design

↓

chip has manufactured

↓



o/p of chip == exp o/p ⇒ no manufacturing defect.

But whether the design will function like half adder?

↳ No

Eventhough there are no manufacturing defect, the circuit will not function like half adder.

Why? The design itself is wrong.

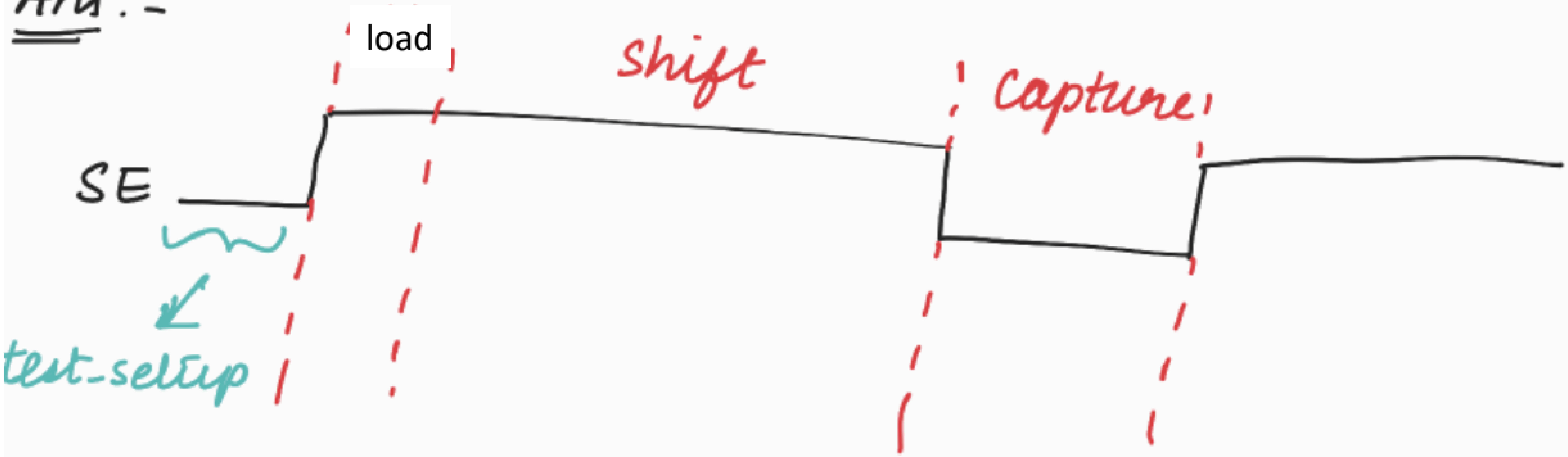
This is the significance of functional verification team.

They will validate the functionality

$$\left.\begin{array}{l} \text{sum} = a+b \\ \text{carry} = ab \end{array}\right\} \rightarrow \text{whether this is correct or not.}$$

Q4:- What is the need of test_setup phase?

Ans:-



The TDRs will be loaded in the test setup phase.